

# AI 协作反思日志

团队：不误正业 日期：2026-05-01

## 1. 本阶段 AI 使用清单

| 任务     | AI工具   | Prompt摘要                         | AI输出质量<br>(1-5) | 人工修改幅度<br>(%) |
|--------|--------|----------------------------------|-----------------|---------------|
| 架构模式对比 | Gemini | 结合校园互助需求，对比MVC分层与微服务架构在10周内的可行性  | 5               | 10%           |
| 架构图生成  | Gemini | 使用 Mermaid 语法绘制分层架构图，要求包含具体的业务模块 | 4               | 20%           |
| 架构决策记录 | Gemini | 生成关于分层架构、JWT认证和积分托管的ADR文档        | 4               | 30%           |
| 技术栈选型  | Gemini | 推荐适合学生团队、低运维成本的后端与移动端技术栈         | 5               | 15%           |

## 2. AI 助力分析（Q1）

**本阶段 AI 最大的贡献是：**快速将抽象的需求转化为可视化的逻辑结构。通过 Mermaid 语法的支持，AI 不仅提供了架构建议，还直接输出了可维护的代码化架构图，极大地缩短了从“概念设计”到“文档产出”的距离。同时，它在 ADR 的编写中提供了标准的工业决策格式，提升了文档的专业性。

**不同 Prompt 策略下 AI 回答质量的差异：**

- **开放式提问**（如“帮我设计架构”）输出结果往往过于通用，甚至偏向复杂的微服务。
- **约束性提问**（如“基于10周交付压力、单体优先、Java技术栈”）能促使 AI 提供更具实践价值、更接地气的具体分层设计和工具推荐。

## 3. AI 误导分析（Q2）

**AI 在架构建议中的典型问题：**

- **问题1：技术堆砌（过度设计）。** AI 初期倾向于推荐分布式锁、服务治理等微服务组件。
- **如何发现：**团队在评估开发复杂度维度时，意识到这些技术超出了成员现有的技能范围，且会拖慢进度。
- **如何修正：**明确告知 AI “团队成员为学生”且“首选单体架构”，引导其回归到简单的分层模型。

## 4. Prompt 改进计划（Q3）

**上阶段（P1）提出的改进措施，验证结果：**P1 提出“需提供具体的业务约束”，本阶段验证效果显著。在明确了10周周期的约束后，AI 不再推荐过度复杂的分布式方案。

**本阶段新的改进措施：**引入“反向提问机制”。在要求 AI 生成方案后，追加一个 Prompt 要求其“指出该方案在10周交付期内最可能遇到的3个工程风险”，以此辅助人工进行风险评估。

## 5. 本阶段的核心工程判断

**决策内容：** 拒绝微服务架构，采用“模块化单体（Monolith）”架构，并使用轻量级的 JWT 替代复杂的身份管理服务。

**为什么 AI 无法做这个决策：** AI 无法准确预估团队成员的“真实技术带宽”以及项目在校内环境下的运维瓶颈。AI 可以计算出理论上的性能最优解，但无法平衡“交付时间”、“人力成本”与“代码可维护性”之间的微妙关系。这种基于项目生存概率的权衡属于人类的工程判断。